

20 cas de données sales

diagnostic et correction



Le cas réaliste, le symptôme visible, la cause réelle, la correction reproductible.

Une donnée sale ne s'annonce jamais elle-même : elle se cache derrière un chiffre qui a l'air normal. Ce catalogue regroupe 20 cas qu'un Data Analyst croise vraiment, classés par famille (structure, complétude, cohérence, valeurs aberrantes, dates, doublons, source). Pour chacun : ce qui cloche à l'œil, pourquoi, et la ligne de code qui corrige.

Structure : la table n'est pas ce qu'elle semble être.

Ici, le problème n'est pas la valeur, c'est la forme même du tableau. On ne le voit jamais dans un aperçu de 10 lignes.

1. En-tête Excel fusionné sur deux lignes

Cas : Un export a un en-tête fusionné sur 2 lignes : « CA 2023 » sur la première, « Q1 Q2 Q3 Q4 » sur la seconde.

Symptôme : après import pandas, les colonnes s'appellent Unnamed: 3, Unnamed: 4...

Diagnostic : la vraie légende est sur la ligne 2, pandas ne lit que la ligne 1 comme en-tête.

Correction : lire les deux lignes comme un en-tête multi-niveau, puis aplatir.

```
df = pd.read_excel(path, header=[0, 1])
df.columns = ['_'.join(str(c) for c in col if 'Unnamed' not in str(c))
              for col in df.columns]
```

2. Format wide au lieu de long (ventes par mois en colonnes)

Cas : Une colonne par mois (jan, fev, mars...) au lieu d'une ligne par mois : le tableau est large, pas long.

Symptôme : impossible de faire un `groupby('mois')`, un `.sum()` additionne des mois différents dans un total illisible.

Diagnostic : la table est au format wide, chaque mois est une colonne, pas une valeur de ligne.

Correction : repasser en long avant toute analyse temporelle avec `df.melt(id_vars='produit', var_name='mois', value_name='ca')`.

3. Clé composite non déclarée (grain ligne de commande)

Cas : `id_commande` se répète car le vrai grain de la table est (`id_commande`, `ligne_produit`), jamais documenté.

Symptôme : un `COUNT(DISTINCT id_commande)` sous-estime largement le nombre réel de lignes de commande.

Diagnostic : la table est au grain ligne, pas commande ; la clé primaire réelle est composite.

Correction : identifier le grain avant tout agrégat, contrôler la clé complète.

À retenir avant tout agrégat

Un chiffre agrégé sur le mauvais grain est faux sans qu'aucune erreur SQL ne le signale.

Le réflexe : `SELECT id_commande, ligne_produit, COUNT(*) FROM t GROUP BY 1,2 HAVING COUNT(*) > 1` révèle si le grain supposé est bien celui de la table.

Complétude : le vide qui ne dit pas son nom.

Un NULL manquant se voit tout de suite. Un NULL déguisé en texte ou en sentinelle, jamais.

4. NaN masqués par du texte (« N/A », « - », « NC »)

Cas : Le vide a été saisi comme une chaîne de caractères au lieu d'un NULL réel, souvent par plusieurs personnes avec des conventions différentes.

Symptôme : `df.isna().sum()` renvoie 0 alors que des lignes sont visiblement vides à l'œil.

Diagnostic : pandas ne reconnaît pas ces chaînes comme des valeurs manquantes par défaut.

Correction : déclarer les valeurs sentinelles textuelles dès l'import avec `pd.read_csv(path, na_values=['N/A', '-', 'NC', 'null'])`.

5. Code sentinelle 9999 / -1 pour « inconnu » PIÈGE CLASSIQUE

Cas : Un ancien système utilisait `age = 999` OU `montant = -1` pour signaler une valeur inconnue, jamais documenté dans aucun dictionnaire de données.

Symptôme : une moyenne d'âge de 130 ans, un chiffre d'affaires moyen négatif qui n'a aucun sens métier.

Diagnostic : la sentinelle a été agrégée comme une vraie valeur, personne ne l'a repérée car il n'y a pas d'erreur technique.

Correction : remplacer explicitement la sentinelle par un NULL avant tout calcul, avec `df['age'] = df['age'].replace(999, pd.NA)`.

Le piège exact

Le mot NULL rassure, mais une sentinelle numérique passe tous les contrôles de complétude classiques.

Bon réflexe

Regarder la distribution (min, max) de chaque colonne numérique avant de moyenner.

Signal envoyé

Tu ne fais jamais confiance à une moyenne sans avoir vérifié les extrêmes.

6. NULL absorbé en zéro après un LEFT JOIN

Cas : Une dimension région incomplète laisse des NULL après une jointure, silencieusement transformés en 0 dans l'agrégat final.

Symptôme : un sous-total de chiffre d'affaires à zéro pour une région qui existe pourtant dans les faits.

Diagnostic : `SUM(COALESCE(ca, 0))` traite « pas de correspondance » comme « vraiment zéro », deux choses différentes.

Correction : compter les NULL post-jointure avant de coalescer, avec `SELECT COUNT(*) FROM t WHERE region_id IS NULL`.

Cohérence : deux colonnes, une seule vérité ?

Le nom d'une colonne ne garantit jamais sa définition exacte. C'est le piège le plus fréquent en reporting financier.

7. montant vs montant_ht jamais définis PIÈGE CLASSIQUE

Cas : Deux colonnes monétaires proches cohabitent dans la table de ventes, sans qu'aucune documentation ne précise laquelle inclut la TVA.

Symptôme : un chiffre d'affaires total 20% plus élevé que celui remonté par la comptabilité.

Diagnostic : `montant` inclut la TVA, `montant_ht` non ; l'agrégat a mélangé les deux sans vérification.

Correction : contrôler l'écart avant tout agrégat, avec `SELECT montant, montant_ht, montant - montant_ht AS ecart FROM ventes LIMIT 20`, puis documenter la définition retenue.

8. Unité de quantité incohérente (kg vs unités)

Cas : L'unité de mesure dépend de la catégorie produit sans qu'aucune colonne dédiée ne le précise : un produit alimentaire en kg, un accessoire en unités, dans la même colonne `quantite`.

Symptôme : un total de « quantité vendue » qui ne veut rien dire, additionnant des kilos et des unités.

Diagnostic : deux unités de mesure hétérogènes cohabitent dans une seule colonne numérique.

Correction : matérialiser une colonne `unite` et vérifier avant toute somme globale, avec `df.groupby('unite')['quantite'].sum()`.

9. Devise mélangée EUR/USD sans colonne dédiée

Cas : Des commandes internationales arrivent en USD, stockées brutes dans la même colonne `montant` que les commandes en EUR.

Symptôme : un chiffre d'affaires total qui varie de +8% d'un mois sur l'autre sans aucune raison métier identifiable.

Diagnostic : aucune colonne devise, aucune conversion vers une devise pivot avant l'agrégat.

Correction : exiger et contrôler une colonne devise systématique, avec `SELECT devise, COUNT(*) FROM commandes GROUP BY devise`, avant toute somme.

Le fil commun de ce chapitre

Une colonne bien nommée n'est pas une colonne bien définie : vérifie toujours avant de sommer.

montant

TVA incluse ou non ?

quantite

Quelle unité, par catégorie ?

devise

EUR partout, vraiment ?

Valeurs aberrantes : le chiffre qui devrait faire tiquer.

Une moyenne ne proteste jamais quand une valeur est absurde. Elle l'avale et donne un résultat faux, poliment.

10. Virgule décimale oubliée à la saisie

Cas : Un prix unitaire saisi ou importé à 15000 au lieu de 15,00 : le séparateur décimal a disparu.

Symptôme : une moyenne de panier multipliée par 10 sur un seul mois, sans explication métier.

Diagnostic : un séparateur décimal a sauté à la saisie ou à l'import, la virgule française lue comme séparateur de milliers.

Correction : isoler les valeurs extrêmes avant publication, avec `df[df['prix_unitaire'] > df['prix_unitaire'].quantile(0.999)]`.

11. Âge négatif ou impossible (date de naissance dans le futur)

Cas : Une date de naissance saisie avec jour et mois inversés tombe après la date du jour, donnant un âge négatif.

Symptôme : un histogramme d'âge affiche une barre à -3 ans, visible au premier coup d'œil sur le graphique.

Diagnostic : aucune borne de plausibilité n'est appliquée à l'ingestion des dates de naissance.

Correction : borner et signaler les valeurs hors plage, avec `df[(df['age'] < 0) | (df['age'] > 110)]`.

12. Échelle de pourcentage incohérente (0-1 vs 0-100)

Cas : Deux sources alimentent la même colonne `taux` avec deux conventions différentes : l'une en 0-1, l'autre en 0-100.

Symptôme : un taux de conversion moyen affiché à 45 au lieu de 0,45%, un ordre de grandeur totalement faux.

Diagnostic : l'échelle n'a jamais été harmonisée ni documentée entre les deux sources.

Correction : normaliser l'échelle à l'ingestion, avec `df['taux'] = df['taux'].apply(lambda x: x/100 if x > 1 else x)`.

Dates : le temps n'est pas un texte.

Une date reste fiable tant que personne ne la stocke en texte libre. Dès qu'elle l'est, elle ment sans le montrer.

13. Formats de dates mélangés JJ/MM/AAAA et ISO PIÈGE CLASSIQUE

Cas : Deux exports de systèmes différents ont été concaténés sans harmoniser le format de date, la colonne restant en texte libre.

Symptôme : un tri chronologique place le 03 janvier après le 12 décembre, sans erreur visible.

Diagnostic : un parseur générique devine le format ligne par ligne sur une colonne au format mixte.

Correction : parser explicitement, jamais laisser deviner sur une colonne mixte.

```
df['date'] = pd.to_datetime(df['date'], format='mixed', dayfirst=True)
assert df['date'].notna().all(), "dates non parsees a verifier"
```

14. Timezone silencieuse (UTC mélangé à l'heure locale)

Cas : Une source stocke ses horodatages en UTC, une autre en heure locale Europe/Paris, sans qu'aucune colonne ne porte l'information de fuseau.

Symptôme : des commandes affichées le jour d'avant ou d'après leur vraie date métier, un pic d'activité à 2h du matin qui n'a pas de sens.

Diagnostic : l'horodatage brut ne dit jamais dans quel fuseau il a été écrit.

Correction : toujours stocker en UTC avec fuseau explicite, convertir seulement à l'affichage, avec `df['ts'].dt.tz_localize('UTC').dt.tz_convert('Europe/Paris')`.

15. Ambiguïté MM/JJ vs JJ/MM sur un export US

Cas : Un export américain écrit ses dates en MM/JJ/AAAA, importé par défaut comme si c'était du JJ/MM/AAAA sans vérification.

Symptôme : un mois entier de commandes semble décalé par rapport au CRM source, sans qu'aucune ligne ne soit rejetée.

Diagnostic : la locale de la source n'a jamais été vérifiée avant l'ingestion.

Correction : déclarer explicitement le format attendu, avec `pd.to_datetime(df['date'], format='%m/%d/%Y')`.

Le fil commun de ce chapitre

Une date en texte libre n'est fiable que si tu as vérifié sa locale d'origine, jamais par défaut.

Le réflexe systématique avant tout import de dates : demander le format et le fuseau à la source, jamais les deviner depuis Python.

Doublons : la même chose, comptée deux fois.

Un doublon ne se voit presque jamais dans un aperçu de 10 lignes. Il se voit dans un total qui ne recoupe pas.

16. Doublons de casse et d'espace (« Paris » / « paris » / « Paris ») PIÈGE CLASSIQUE

Cas : Aucune normalisation de casse ni de trim n'a été appliquée à la saisie ou à l'import, laissant trois graphies pour une seule ville réelle.

Symptôme : un TOP 10 des villes affiche Paris deux fois, avec des volumes divisés entre les variantes.

Diagnostic : trois valeurs textuelles distinctes pour SQL ou pandas, une seule ville pour le métier.

Correction : normaliser systématiquement avant tout regroupement, avec `df['ville'] = df['ville'].str.strip().str.title()`.

17. Doublons partiels après fusion de deux fichiers clients

Cas : Les mêmes clients existent avec des identifiants légèrement différents entre deux systèmes CRM, souvent une différence de casse ou d'espace dans l'email.

Symptôme : un nombre de clients uniques supérieur au total attendu par le CRM de référence.

Diagnostic : la clé de rapprochement diffère légèrement entre les deux sources, jamais normalisée avant le merge.

Correction : normaliser la clé de jointure avant tout merge, avec `df['email'] = df['email'].str.strip().str.lower()`.

18. Clés dupliquées après un merge many-to-many non voulu ÉLIMINATOIRE

Cas : Une jointure entre ventes et catégories utilise une clé produit non unique côté droit, à cause d'un historique de prix gardant 3 lignes par produit.

Symptôme : un chiffre d'affaires total multiplié par 3 après la jointure, sans aucune erreur SQL ou pandas.

Diagnostic : la table de droite contient plusieurs lignes par clé de jointure, le grain n'a jamais été vérifié.

Correction : vérifier l'unicité de la clé avant tout merge, avec `df_categorie['produit_id'].duplicated().sum()`, puis comparer `len(df)` avant et après la jointure.

Pourquoi ce cas élimine

Un merge qui multiplie les lignes gonfle un chiffre sans produire aucune erreur technique visible.

Grain

Une ligne, un produit?

Cardinalité

Clé unique côté droit?

Contrôle

Lignes avant/après merge?

Source et typage : le fichier ment même l'analyse.

Ici, l'erreur arrive avant la première ligne de code, dès l'import.

19. Encodage cassé (« Ã© » au lieu de « é »)

Cas : Un fichier encodé en UTF-8 est lu comme s'il était en Latin-1 (ou l'inverse), sans qu'aucune vérification d'encodage n'ait été faite à la source.

Symptôme : des noms de produits illisibles, des filtres textuels sur les accents qui ne matchent plus rien.

Diagnostic : l'encodage de lecture n'a jamais été déclaré explicitement, pandas devine et se trompe.

Correction : déclarer explicitement l'encodage à la lecture, avec `pd.read_csv(path, encoding='utf-8')`, vérifier l'encodage source avant tout.

20. Nombres en texte avec espace insécable (« 1 234,56 »)

Cas : Un export européen utilise un espace insécable comme séparateur de milliers et une virgule comme séparateur décimal, un format que pandas ne reconnaît pas nativement.

Symptôme : la colonne `montant` reste de type objet (texte), une somme concatène des chaînes au lieu d'additionner des nombres.

Diagnostic : le format numérique européen n'est jamais reconnu par défaut par un parseur anglo-saxon.

Correction : nettoyer puis convertir explicitement avant tout calcul.

```
df['montant'] = (df['montant']
                .str.replace('\xa0', '', regex=False)
                .str.replace(',', '.', regex=False)
                .astype(float))
```

20 sur 20. Sur le terrain, ces 20 cas reviennent plus souvent que n'importe quelle question d'entretien : ils se cachent dans des chiffres qui, au premier regard, paraissent parfaitement normaux. Le seul réflexe qui protège vraiment : ne jamais publier un agrégat sans avoir vérifié le grain, la complétude réelle et l'unité de ce que tu additionnes.

Fait main, en solo.

Garde ce catalogue sous la main : 20 nettoyages prêts à réappliquer sur ton prochain fichier. Si un cas te parle particulièrement, dis-le moi en commentaire, c'est là que je pioche mes prochains catalogues.

Dylan

